Informe post-incidente — Simulacro de desastre (Unidad 6)

Food Store · PostgreSQL 18 · Moran, Joaquín Leandro

Base principal `foodstore`; restauración en `foodstore_restore_test`. Las evidencias están en `evidencias/`.

Respaldo utilizado

Dato	Valor
Archivo	`backups/foodstore_<fecha y hora>.dump`, el último que generó `backup_foodstore.sh` (listado en `evidencias/respaldos.txt`)
Fecha y hora del respaldo	La del nombre del archivo y de la línea `INICIO` de `logs/backup.log` (copiada en `evidencias/respaldos.txt`)
Tamaño	7,9 MB (8.243.372 bytes)
Verificación de legibilidad	`pg_restore --list`: 230 objetos (`logs/backup.log`, línea `OK`); el respaldo tardó 2 s

Tiempos, RPO y RTO

	Previsto	Observado
RTO (desde `createdb` hasta el fin de las validaciones)	≤ 15 min	6,666 s (de los cuales `createdb` + `pg_restore`: 5,268 s)
RPO (antigüedad del respaldo al restaurar)	≤ 24 h, con el respaldo diario	5 min 40 s

RTO es cuánto tarda en volver a estar disponible la base. **RPO** es cuántos datos se pueden perder: todo lo que se escribió después del último respaldo. En el simulacro, el RPO observado es corto porque el respaldo se hizo justo antes. En un desastre real, con el respaldo diario de las 02:00, una falla a las 23:59 perdería casi 22 horas de ventas.

Validaciones sobre `foodstore_restore_test`

Tabla	Filas antes del respaldo	Filas restauradas	¿Coinciden?
categoria	5	5	Sí
cliente	20.005	20.005	Sí
producto	50.010	50.010	Sí
pedido	200.009	200.009	Sí
detalle_pedido	399.997	399.997	Sí

Productos inválidos: 0 · Detalles inválidos: 0 · Resultado: **Validaciones: OK** (`evidencias/restore_test_*.txt`)

Problema encontrado

La primera corrida del simulacro dio **«Validaciones: FALLARON»** con todos los conteos iguales (5 contra 5, 20.005 contra 20.005…). La causa: en Windows, `psql` termina cada línea con `\r\n`, y el script comparaba `5\r` con `5`. No era un problema del respaldo sino de la validación. Se corrigió quitando el `\r`

antes de comparar (`tr -d '\r'` en `restore_test.sh`) y se volvió a correr: **OK**. La corrida que falló quedó como evidencia en `evidencias/restore_test_antes_de_corregir_crlf.txt`.

Después, al programar el respaldo, la tarea quedaba creada pero no hacía nada, y no dejaba ningún mensaje. Había dos causas. El comando de la spec ponía comillas simples dentro de `/TR`, y Windows no las reconoce. Además, Windows no inicia una tarea programada si la notebook está a batería. Se resolvió con `programar_respaldo.cmd`, que crea la tarea, permite que corra a batería y llama a `respaldo_tarea.cmd`. Este guarda toda la salida en `logs/tarea_programada.log`, así que un fallo a las 02:00 ya no pasa inadvertido. Con eso, la tarea generó el respaldo con código de salida 0 (`evidencias/tarea_programada.txt`).

Antes, en la Unidad 5, había pasado algo parecido: en Windows, `python3` es un acceso directo a la Microsoft Store y no el Python instalado, así que los scripts que lo usaban fallaban. Ahora buscan el primer Python que funciona de verdad.

Mejora propuesta

Restaurar en paralelo con `pg_restore --jobs`. El formato custom lo permite: índices y restricciones se recrean en varios procesos. Medido con `JOBS=4 ./restore_test.sh`: RTO de 5,014 s (`createdb` + `pg_restore`: 3,880 s), contra 6,666 s con un solo proceso. Para bajar el **RPO** por debajo de 24 h, el paso siguiente sería sumar archivado continuo de WAL (recuperación a un punto en el tiempo).

Trabajo asistido con OpenCode

OpenCode generó una versión propia de los cuatro scripts a partir de `SPEC_U6.md` y `AGENTS.md`, en `opencode/`. No se ejecutó: se leyó línea por línea y se comparó contra los scripts de referencia, revisando los cinco puntos de `prompts/opencode_generar_scripts.md`. Lo aceptado y lo descartado está en `evidencias/revision_opencode.md`. Por ejemplo, su `restore_test.sh` consultaba tablas que no existen (`products`, `orders`, `order_details`, `customers`): la spec no nombra las tablas y OpenCode las inventó. Además arrancaba el cronómetro antes de contar las filas de la base principal, no justo antes de `createdb`. Se descartó. Lo que sí se tomó fue el porcentaje de filas modificadas desde el último `ANALYZE` en el chequeo de salud. Y al comparar apareció que la referencia exigía un mínimo de 50 filas modificadas que la spec no decía: se agregó a la spec. Los scripts finales se ejecutaron primero a mano y después desde la tarea programada (`evidencias/tarea_programada.txt`).